# Guess X - REDIS_HA


- 1 Redis High Availability Configuration for Guess X
- 2 This file documents the Redis HA setup options
  - 2.1 Option 1: Redis Sentinel (Recommended for simpler HA)
    - 2.1.1 Docker Compose for Sentinel (Development)
    - 2.1.2 Master Configuration (redis/master.conf)
    - 2.1.3 Replica Configuration (redis/replica.conf)
    - 2.1.4 Sentinel Configuration (redis/sentinel.conf)
  - 2.2 Option 2: Redis Cluster (For horizontal scaling)
    - 2.2.1 Docker Compose for Cluster (Production)
    - 2.2.2 Cluster Configuration (redis/cluster.conf)
    - 2.2.3 Initialize Cluster
  - 2.3 Application Configuration
    - 2.3.1 Environment Variables for Sentinel
    - 2.3.2 Environment Variables for Cluster
    - 2.3.3 NestJS Redis Provider for Sentinel
    - 2.3.4 NestJS Redis Provider for Cluster
  - 2.4 Kubernetes Deployment for Redis HA
    - 2.4.1 Sentinel Deployment
  - 2.5 Health Checks and Monitoring
    - 2.5.1 Redis Health Check Script
  - 2.6 Failover Testing
    - 2.6.1 Test Sentinel Failover
    - 2.6.2 Test Cluster Failover
  - 2.7 Production Recommendations


# 1 Redis High Availability Configuration for Guess X

# 2 This file documents the Redis HA setup options

## 2.1 Option 1: Redis Sentinel (Recommended for simpler HA)

### 2.1.1 Docker Compose for Sentinel (Development)

```yaml
# docker-compose.sentinel.yml
version: '3.8'

services:
  redis-master:
    image: redis:7-alpine
    command: redis-server /usr/local/etc/redis/redis.conf
    volumes:
      - ./redis/master.conf:/usr/local/etc/redis/redis.conf
      - redis-master-data:/data
    networks:
      - redis-net
    deploy:
      resources:
        limits:
          memory: 512M
        reservations:
          memory: 256M

  redis-replica-1:
    image: redis:7-alpine
    command: redis-server /usr/local/etc/redis/redis.conf
    volumes:
      - ./redis/replica.conf:/usr/local/etc/redis/redis.conf
      - redis-replica1-data:/data
    depends_on:
      - redis-master
    networks:
      - redis-net
    deploy:
      resources:
        limits:
          memory: 512M
        reservations:
          memory: 256M

  redis-replica-2:
    image: redis:7-alpine
    command: redis-server /usr/local/etc/redis/redis.conf
    volumes:
      - ./redis/replica.conf:/usr/local/etc/redis/redis.conf
      - redis-replica2-data:/data
    depends_on:
```

```
      - redis-master
    networks:
      - redis-net
    deploy:
      resources:
        limits:
          memory: 512M
        reservations:
          memory: 256M

  sentinel-1:
    image: redis:7-alpine
    command: redis-sentinel /usr/local/etc/redis/sentinel.conf
    volumes:
      - ./redis/sentinel.conf:/usr/local/etc/redis/sentinel.conf
    depends_on:
      - redis-master
      - redis-replica-1
      - redis-replica-2
    networks:
      - redis-net

  sentinel-2:
    image: redis:7-alpine
    command: redis-sentinel /usr/local/etc/redis/sentinel.conf
    volumes:
      - ./redis/sentinel.conf:/usr/local/etc/redis/sentinel.conf
    depends_on:
      - redis-master
      - redis-replica-1
      - redis-replica-2
    networks:
      - redis-net

  sentinel-3:
    image: redis:7-alpine
    command: redis-sentinel /usr/local/etc/redis/sentinel.conf
    volumes:
      - ./redis/sentinel.conf:/usr/local/etc/redis/sentinel.conf
    depends_on:
      - redis-master
      - redis-replica-1
      - redis-replica-2
    networks:
      - redis-net

networks:
  redis-net:
    driver: bridge
```

```
volumes:
  redis-master-data:
  redis-replica1-data:
  redis-replica2-data:
```

### 2.1.2 Master Configuration (redis/master.conf)

```
bind 0.0.0.0
port 6379
appendonly yes
appendfsync everysec
save 900 1
save 300 10
save 60 10000
maxmemory 256mb
maxmemory-policy allkeys-lru
replica-announce-ip redis-master
replica-announce-port 6379
```

### 2.1.3 Replica Configuration (redis/replica.conf)

```
bind 0.0.0.0
port 6379
replicaof redis-master 6379
appendonly yes
appendfsync everysec
save 900 1
save 300 10
save 60 10000
maxmemory 256mb
maxmemory-policy allkeys-lru
replica-announce-ip redis-replica-1
replica-announce-port 6379
```

### 2.1.4 Sentinel Configuration (redis/sentinel.conf)

```
port 26379
sentinel monitor guessx-master redis-master 6379 2
sentinel down-after-milliseconds guessx-master 5000
sentinel failover-timeout guessx-master 60000
sentinel parallel-syncs guessx-master 1
```

```
sentinel resolve-hostnames yes
sentinel announce-hostnames yes
```

---

## 2.2 Option 2: Redis Cluster (For horizontal scaling)

### 2.2.1 Docker Compose for Cluster (Production)

```yaml
# docker-compose.cluster.yml
version: '3.8'

services:
  redis-node-0:
    image: redis:7-alpine
    command: redis-server /usr/local/etc/redis/redis-cluster.conf
    volumes:
      - ./redis/cluster.conf:/usr/local/etc/redis/redis-cluster.conf
      - redis-node0-data:/data
    ports:
      - "7000:7000"
      - "17000:17000"
    networks:
      - redis-cluster-net

  redis-node-1:
    image: redis:7-alpine
    command: redis-server /usr/local/etc/redis/redis-cluster.conf
    volumes:
      - ./redis/cluster.conf:/usr/local/etc/redis/redis-cluster.conf
      - redis-node1-data:/data
    ports:
      - "7001:7001"
      - "17001:17001"
    networks:
      - redis-cluster-net

  redis-node-2:
    image: redis:7-alpine
    command: redis-server /usr/local/etc/redis/redis-cluster.conf
    volumes:
      - ./redis/cluster.conf:/usr/local/etc/redis/redis-cluster.conf
      - redis-node2-data:/data
    ports:
      - "7002:7002"
      - "17002:17002"
    networks:
```

```
      - redis-cluster-net

  redis-node-3:
    image: redis:7-alpine
    command: redis-server /usr/local/etc/redis/redis-cluster.conf
    volumes:
      - ./redis/cluster.conf:/usr/local/etc/redis/redis-cluster.conf
      - redis-node3-data:/data
    ports:
      - "7003:7003"
      - "17003:17003"
    networks:
      - redis-cluster-net

  redis-node-4:
    image: redis:7-alpine
    command: redis-server /usr/local/etc/redis/redis-cluster.conf
    volumes:
      - ./redis/cluster.conf:/usr/local/etc/redis/redis-cluster.conf
      - redis-node4-data:/data
    ports:
      - "7004:7004"
      - "17004:17004"
    networks:
      - redis-cluster-net

  redis-node-5:
    image: redis:7-alpine
    command: redis-server /usr/local/etc/redis/redis-cluster.conf
    volumes:
      - ./redis/cluster.conf:/usr/local/etc/redis/redis-cluster.conf
      - redis-node5-data:/data
    ports:
      - "7005:7005"
      - "17005:17005"
    networks:
      - redis-cluster-net

networks:
  redis-cluster-net:
    driver: bridge

volumes:
  redis-node0-data:
  redis-node1-data:
  redis-node2-data:
  redis-node3-data:
  redis-node4-data:
  redis-node5-data:
```

### 2.2.2 Cluster Configuration (redis/cluster.conf)

```
bind 0.0.0.0
port 7000
cluster-enabled yes
cluster-config-file nodes.conf
cluster-node-timeout 5000
appendonly yes
appendfsync everysec
save 900 1
save 300 10
save 60 10000
maxmemory 256mb
maxmemory-policy allkeys-lru
```

### 2.2.3 Initialize Cluster

```
# Create cluster after containers are running
docker exec -it redis-node-0 redis-cli --cluster create \
  redis-node-0:7000 \
  redis-node-1:7001 \
  redis-node-2:7002 \
  redis-node-3:7003 \
  redis-node-4:7004 \
  redis-node-5:7005 \
  --cluster-replicas 1
```

---

## 2.3 Application Configuration

### 2.3.1 Environment Variables for Sentinel

```
# Redis Sentinel Configuration
REDIS_SENTINEL_HOSTS=sentinel-1:26379,sentinel-2:26379,sentinel-3:26379
REDIS_SENTINEL_MASTER_NAME=guessx-master
REDIS_SENTINEL_ROLE=master
```

### 2.3.2 Environment Variables for Cluster

```
# Redis Cluster Configuration
```

```
REDIS_CLUSTER_NODES=redis-node-0:7000,redis-node-1:7001,redis-node-2:7002,redis-n
```

### 2.3.3 NestJS Redis Provider for Sentinel

```
// src/modules/common/redis-sentinel.provider.ts
import { Provider } from '@nestjs/common';
import { createClient, RedisClientType, SentinelIterator } from 'redis';

export const REDIS_SENTINEL_CLIENT = 'REDIS_SENTINEL_CLIENT';

export const redisSentinelProvider: Provider = {
  provide: REDIS_SENTINEL_CLIENT,
  useFactory: async () => {
    const sentinelHosts = process.env.REDIS_SENTINEL_HOSTS?.split(',') ||
        ['localhost:26379'];
    const masterName = process.env.REDIS_SENTINEL_MASTER_NAME || 'guessx-
        master';

    // Create sentinel iterator
    const sentinelIterator = new SentinelIterator({
      sentinels: sentinelHosts.map(h => {
        const [host, port] = h.split(':');
        return { host, port: parseInt(port) };
      }),
      name: masterName,
      role: 'master',
    });

    // Get master address
    const master = await sentinelIterator.next();
    if (!master) {
      throw new Error('No Redis master found');
    }

    // Create client connected to master
    const client = createClient({
      socket: {
        host: master.host,
        port: master.port,
      },
    });

    await client.connect();

    // Handle master switch
    client.on('error', async (err) => {
      console.error('Redis client error:', err);
```

```
    });

    return client;
  },
};
```

### 2.3.4 NestJS Redis Provider for Cluster

```
// src/modules/common/redis-cluster.provider.ts
import { Provider } from '@nestjs/common';
import { createCluster, RedisClusterType } from 'redis';

export const REDIS_CLUSTER_CLIENT = 'REDIS_CLUSTER_CLIENT';

export const redisClusterProvider: Provider = {
  provide: REDIS_CLUSTER_CLIENT,
  useFactory: async () => {
    const nodes = (process.env.REDIS_CLUSTER_NODES || 'localhost:7000')
      .split(',')
      .map(n => {
        const [host, port] = n.split(':');
        return { host, port: parseInt(port) };
      });

    const cluster = createCluster({
      rootNodes: nodes,
      defaults: {
        enableReadyCheck: true,
        maxRetriesPerRequest: 3,
      },
    });

    await cluster.connect();

    cluster.on('error', (err) => {
      console.error('Redis cluster error:', err);
    });

    return cluster;
  },
};
```

## 2.4 Kubernetes Deployment for Redis HA

### 2.4.1 Sentinel Deployment

```yaml
# k8s/redis-sentinel.yaml
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: redis
  namespace: takelow
spec:
  serviceName: redis
  replicas: 3
  selector:
    matchLabels:
      app: redis
  template:
    metadata:
      labels:
        app: redis
    spec:
      containers:
      - name: redis
        image: redis:7-alpine
        command: ["redis-server"]
        args: ["--appendonly", "yes", "--maxmemory", "256mb", "--maxmemory-
        policy", "allkeys-lru"]
        ports:
        - containerPort: 6379
        volumeMounts:
        - name: redis-data
          mountPath: /data
        resources:
          requests:
            memory: "256Mi"
            cpu: "100m"
          limits:
            memory: "512Mi"
            cpu: "500m"
      - name: sentinel
        image: redis:7-alpine
        command: ["redis-sentinel"]
        args: ["/etc/redis/sentinel.conf"]
        ports:
        - containerPort: 26379
        volumeMounts:
        - name: sentinel-config
          mountPath: /etc/redis
  volumeClaimTemplates:
  - metadata:
      name: redis-data
```

```
    spec:
      accessModes: ["ReadWriteOnce"]
      resources:
        requests:
          storage: 1Gi
```

---

## 2.5 Health Checks and Monitoring

### 2.5.1 Redis Health Check Script

```
#!/bin/bash
# scripts/redis-health-check.sh

check_sentinel() {
    local sentinel_host="${1:-localhost}"
    local sentinel_port="${2:-26379}"
    local master_name="${3:-guessx-master}"

    local master_info=$(redis-cli -h "$sentinel_host" -p "$sentinel_port"
        SENTINEL MASTER "$master_name" 2>/dev/null)

    if [[ -z "$master_info" ]]; then
        echo "FAIL: Cannot get master info from Sentinel"
        return 1
    fi

    local status=$(echo "$master_info" | grep "flags" | awk '{print $2}')
    if [[ "$status" != *"master"* ]]; then
        echo "FAIL: Master not found or not healthy"
        return 1
    fi

    echo "OK: Sentinel reports master healthy"
    return 0
}

check_cluster() {
    local node="${1:-localhost:7000}"

    local cluster_info=$(redis-cli -c -h "${node%:*}" -p "${node#*:}" CLUSTER
        INFO 2>/dev/null)

    if [[ -z "$cluster_info" ]]; then
        echo "FAIL: Cannot connect to cluster"
        return 1
```

```
    fi

    local state=$(echo "$cluster_info" | grep "cluster_state" | cut -d: -f2)
    if [[ "$state" != "ok" ]]; then
        echo "FAIL: Cluster state is $state"
        return 1
    fi

    echo "OK: Cluster is healthy"
    return 0
}

# Usage
if [[ "$1" == "sentinel" ]]; then
    check_sentinel "$2" "$3" "$4"
elif [[ "$1" == "cluster" ]]; then
    check_cluster "$2"
else
    echo "Usage: $0 {sentinel|cluster} [host] [port] [master_name]"
fi
```

---

## 2.6 Failover Testing

### 2.6.1 Test Sentinel Failover

```
# Simulate master failure
docker kill redis-master

# Watch Sentinel logs
docker logs -f sentinel-1

# Verify new master
redis-cli -h sentinel-1 -p 26379 SENTINEL MASTER guessx-master
```

### 2.6.2 Test Cluster Failover

```
# Simulate node failure
docker kill redis-node-0

# Check cluster status
redis-cli -h redis-node-1 -p 7001 CLUSTER INFO
redis-cli -h redis-node-1 -p 7001 CLUSTER NODES
```

## 2.7 Production Recommendations

1. **Use Redis Sentinel for simpler HA** - Automatic failover, minimal config
2. **Use Redis Cluster for horizontal scaling** - Sharding across multiple nodes
3. **Enable AOF persistence** - `appendonly yes` with `appendfsync everysec`
4. **Set appropriate memory limits** - `maxmemory 256mb` with `allkeys-lru` policy
5. **Monitor key metrics**:
   - `used_memory` / `maxmemory`
   - `connected_clients`
   - `replication_lag` (for replicas)
   - `cluster_state` (for cluster)
6. **Regular backup** - Use RDB snapshots or AOF rewrites
7. **Test failover regularly** - Monthly failover drills